

Parallel and Distributed Systems

Chapter 1: Introduction to Parallel and Distributed Systems

Prof. Dr. Martin Sträßer



Copyright & Acknowledgments

- All materials (incl. lecture and exercise slides, recordings, exercise tasks and sheets, programming tasks) are provided for personal use only. Reproducing, copying, uploading, sharing, or providing materials to third parties is not permitted



Overview

- Concurrency and Parallelism
- Structure of Concurrent Programs

Concurrency and Parallelism

Serial Execution

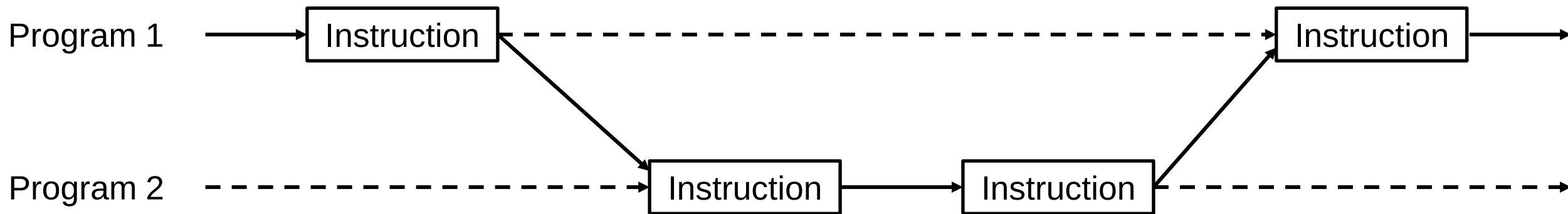
- Until 1980s, computers ran one program at a time, instruction after instruction, in a strictly sequential manner
- This is called *serial* or *sequential execution*



- (Only) key performance indicator for CPUs was frequency in Hz (instructions per second)
- Intel 80386 processor was one of first processors that allowed to interrupt a program in order to work on another
- This allows *time sharing* between different programs

Concurrent Execution

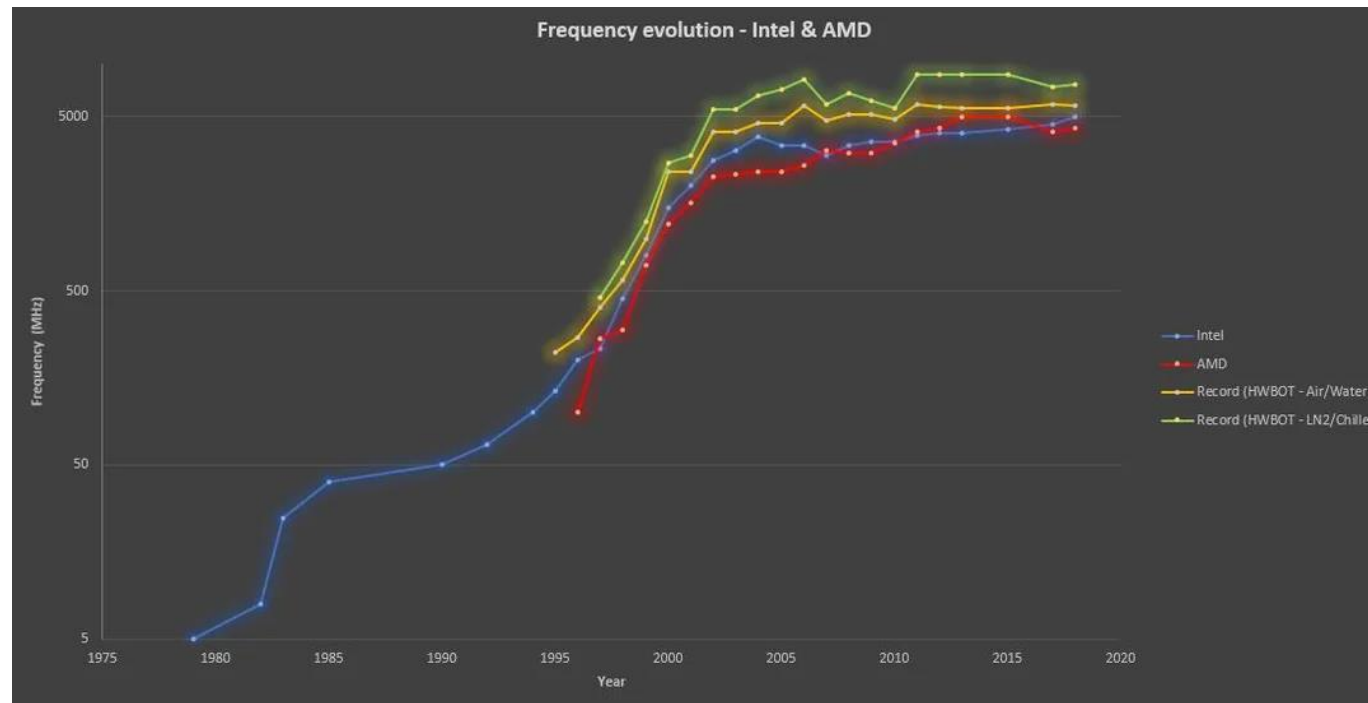
- Switching between and resuming programs allows us manage multiple programs *at the same time, but not simultaneously*
- This is called *concurrent execution*



- If the instruction processing and process switching takes place at a high frequency, it *seems* that multiple programs are executed *in parallel*

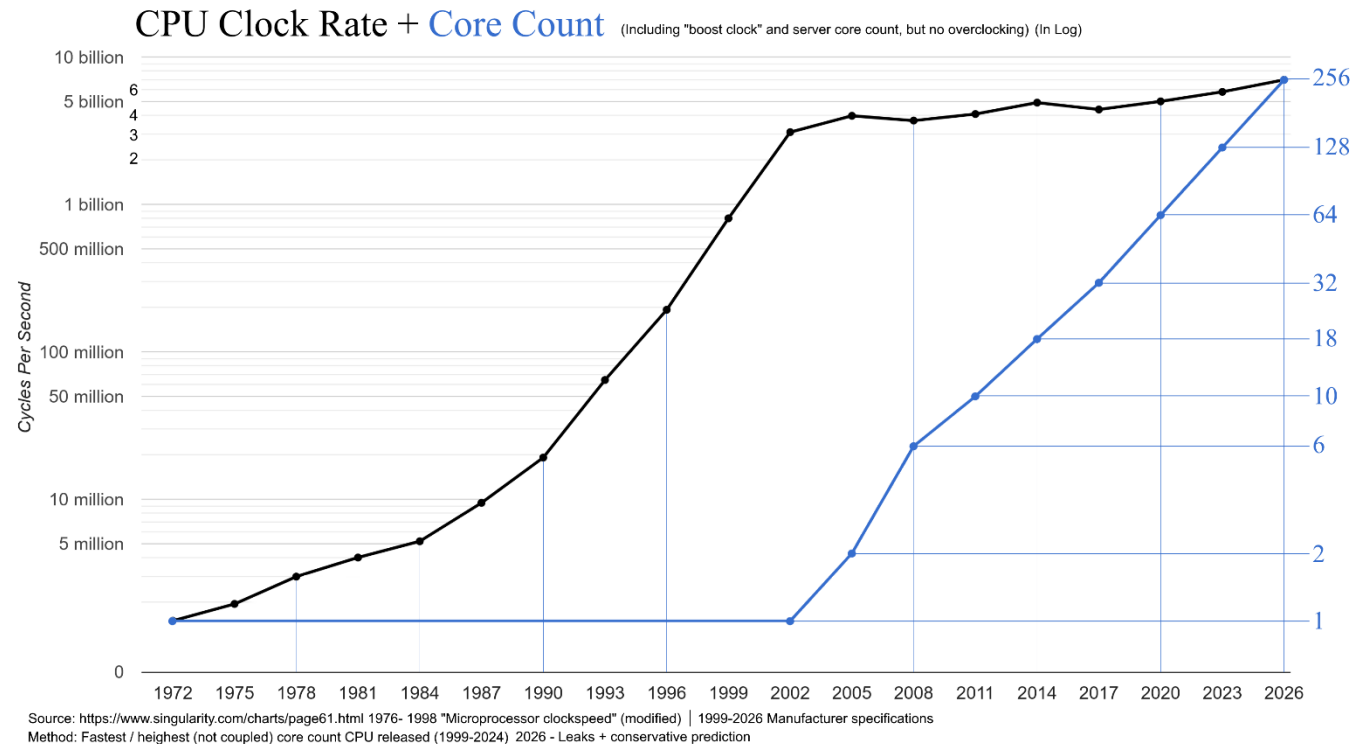
Discussion

- Which factors prevent us from increasing the frequency (instructions per second) of processors?
- What could we do else to handle more programs at the same time/complete programs faster?



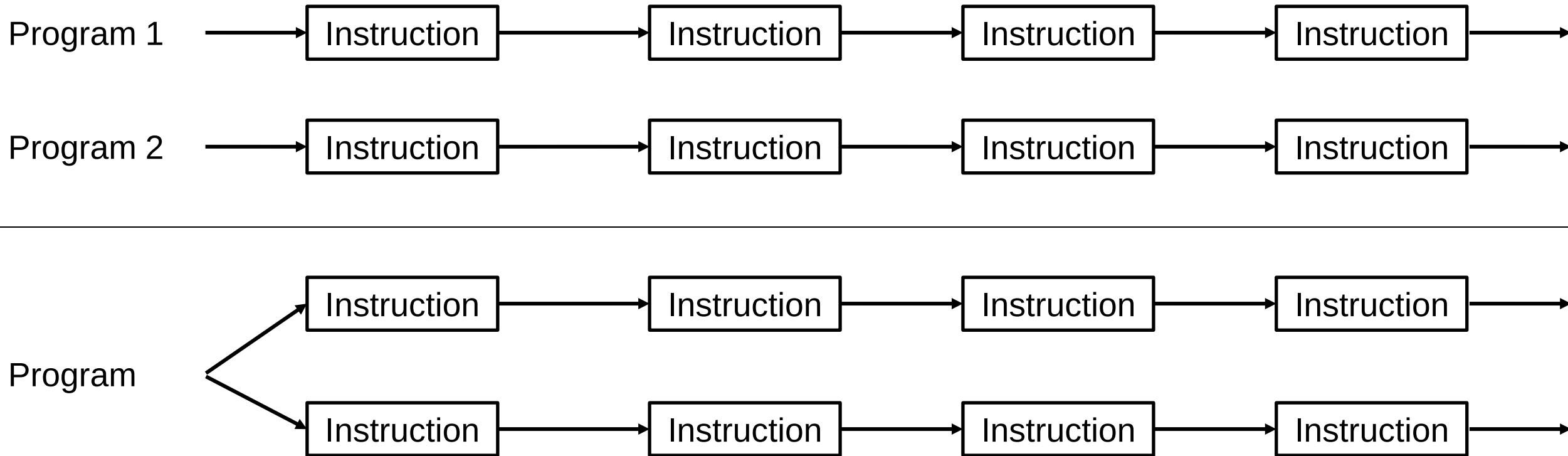
Logical Processors

- Main problems: Heat and energy consumption
- Solution: Multiple “processors” within a single CPU
- We call these multiple processors within a single CPU *logical processors or cores*



Parallel Execution

- Given that we have a multi-core system, we can process multiple instructions from a single or multiple programs at the same time
- This is called *parallel* or *true parallel execution*





Concurrency vs. Parallelism

- The terms concurrency and parallelism are sometimes used interchangeably especially in online sources
- Differences
 - Concurrency refers to the ability of managing multiple tasks/programs at the same time. It makes no statement whether these tasks/programs are executed simultaneously.
 - Parallelism means executing multiple tasks/programs at the same time simultaneously
 - Concurrency therefore refers more to the design aspect, parallelism refers to the execution aspect



Concurrency vs. Parallelism

- For high-level programming languages the term concurrent programming is more precise than parallel programming
- Example: Java
 - Core language concepts: keyword synchronized, classes Thread, Lock, ExecutorService, and CompletableFuture
 - All these concepts allow for concurrent programming, not explicitly parallel programming
- True parallel execution always requires a multi-core system and, in the end, the operating system decides about the usage of cores
- In a multi-core system, it is very likely that a concurrent program is executed on multiple cores in parallel, that is why the term parallel programming makes sense as well

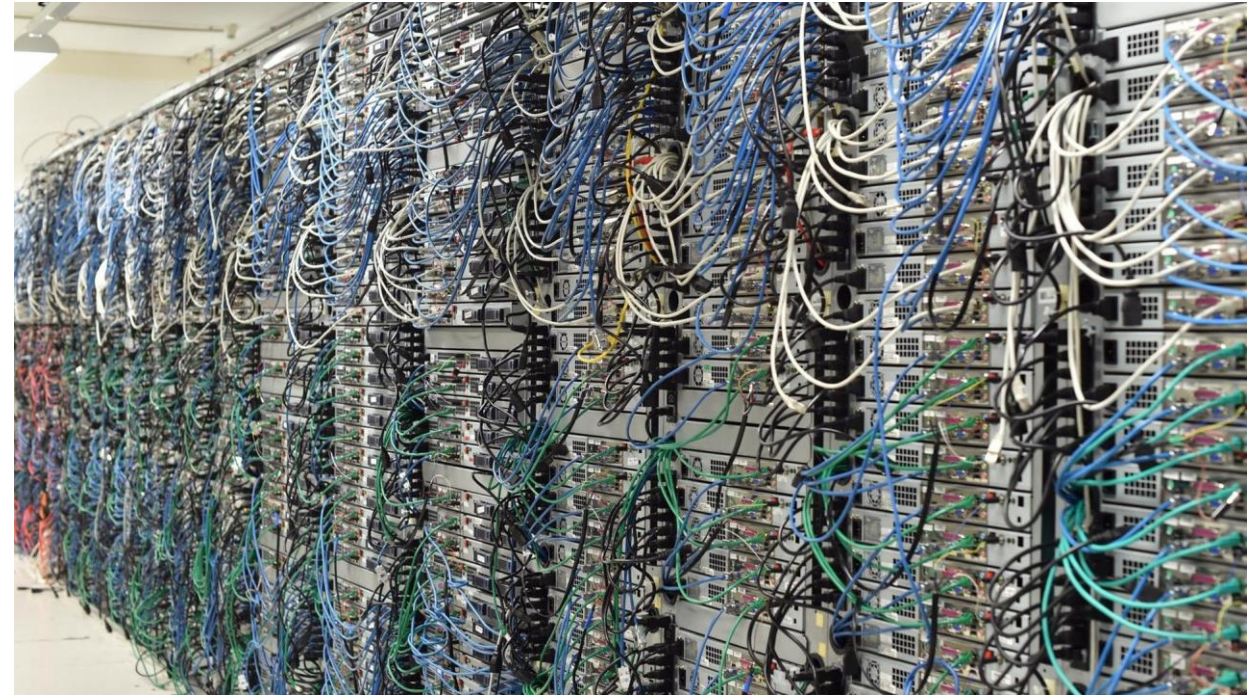


Discussion

- Which factors prevent us from using arbitrarily many cores on one CPU?
- What else could we do to handle more programs at the same time/complete programs faster?

Distributed Systems

- Problems
 - Chip and electronical limitations
 - Connected devices (e.g., memory)
 - Programmability, limited use cases (Task: Think about a useful program that uses 100+ cores)
- Solution: Use multiple machines, use a *distributed system*





Discussion

- Which challenges arise when we split a program to multiple machines?



Selected Challenges in Distributed Systems

- Communication (via Network)
- Coordination
- (Time) Synchronization
- Task Assignment/Load Balancing
- Fault Tolerance
- Operational Complexity (Monitoring, Machine Configuration, ...)



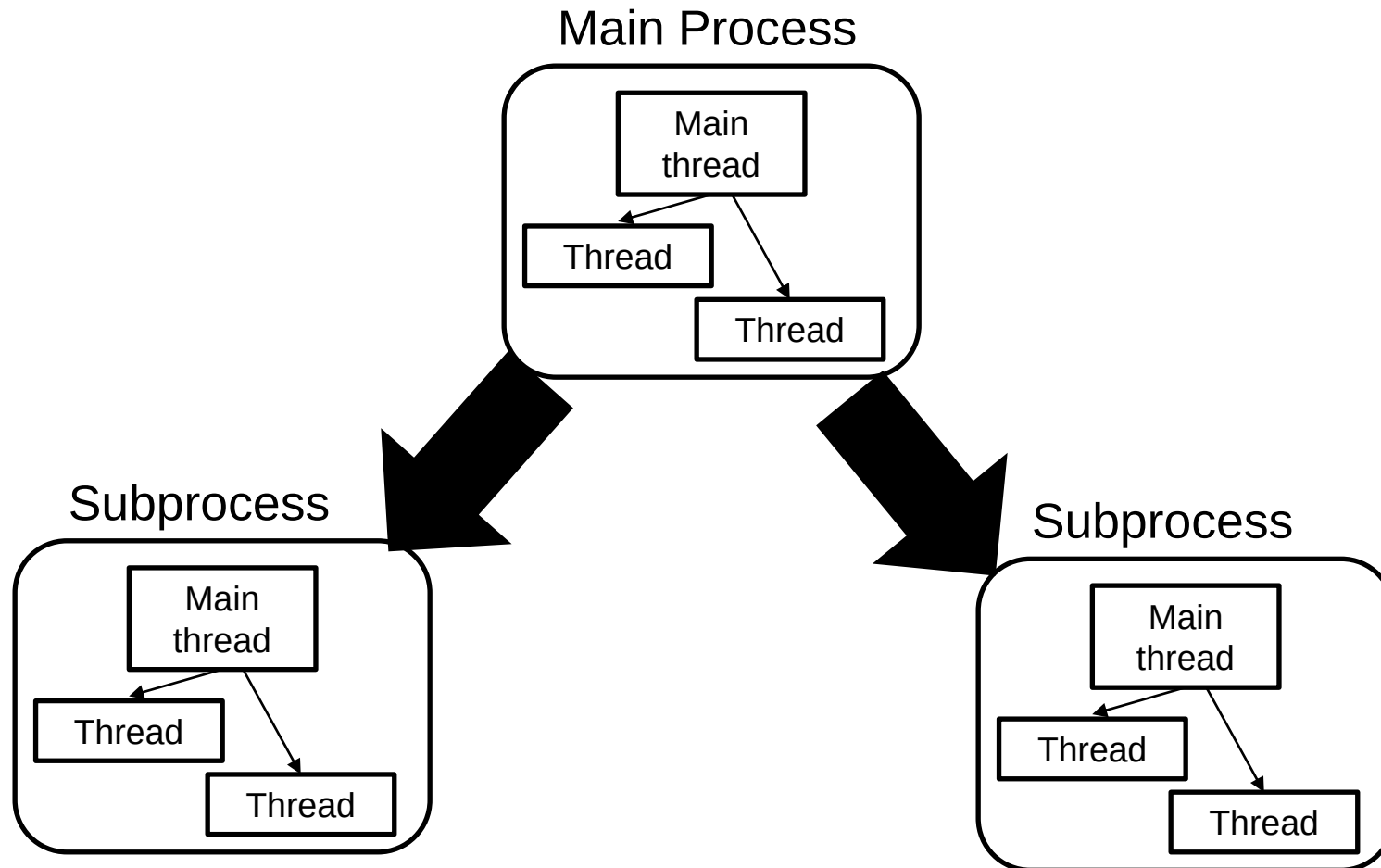
Parallel vs. Distributed Systems

Parallel Systems	Distributed Systems
Multiple processes or threads running on a single machine with a multi-core processor	An application is executed on multiple independent machines
Communication between processes/threads is fast	Communication between machines via network is comparatively slow
Certain resources (e.g., memory) can be shared between different processes/threads	No shared resources between machines
Scalability limited by the performance/resources of one machine	Virtually infinite scalability (cloud computing)

Structure of Concurrent Programs



Structure of Concurrent Programs





Structure of Concurrent Programs

- Any modern program always consists of a main process running a main thread
- To make a program concurrent, we may
 - Spawn multiple threads inside a process and/or
 - Spawn subprocesses
- Subprocesses also have a main thread and may spawn other threads
- Threads are the entities that are assigned to a core by the operating system
- That is why both threads and processes can theoretically enable parallel execution



Differences Between Threads and Processes

- We will cover the differences between threads and processes in detail later in this course
- Roughly speaking
 - Two processes are independent resource containers with own address spaces, environments, etc.
 - Two threads within a process share common resources (e.g., the address space)
 - Threads are lightweight, but may require thoughtful synchronization and provide less isolation (e.g., in terms of security aspects)
 - Processes are heavier and should typically be longer living

Threads and Processes in Action

